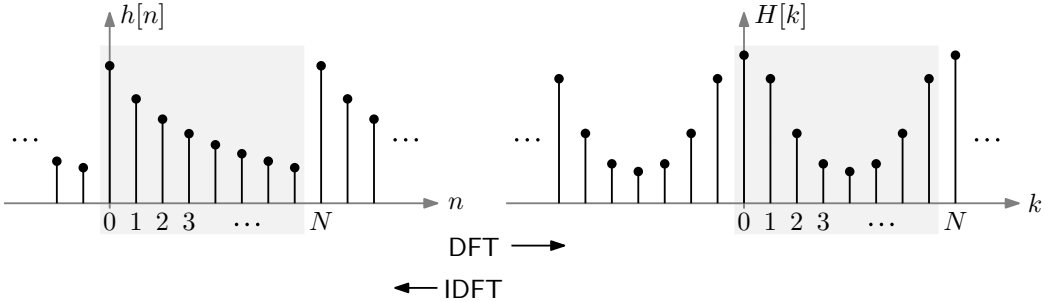
Fast Fourier transform (FFT)

An example of how to compute things quickly

Herman Kamper


$h[n]$
n
0 1 2 3 $\cdots$ N
$H[k]$
k
0 1 2 3 $\cdots$ N
DFT
IDFT

Fast Fourier transform (FFT)

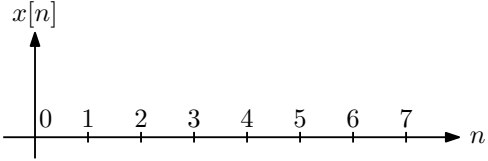
$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

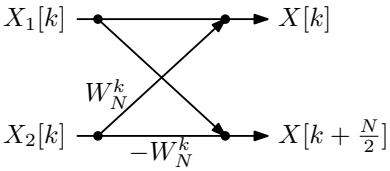
Complex multiplications:

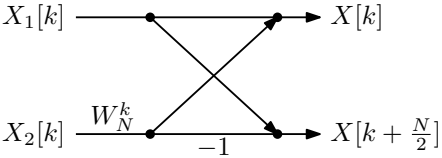
The FFT and DFT lead to precisely the same result. They differ only in the way that they are calculated.

Decimation in time

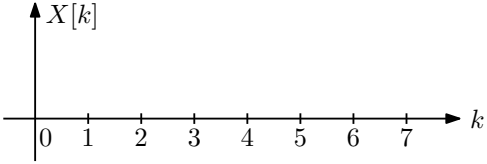
$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$



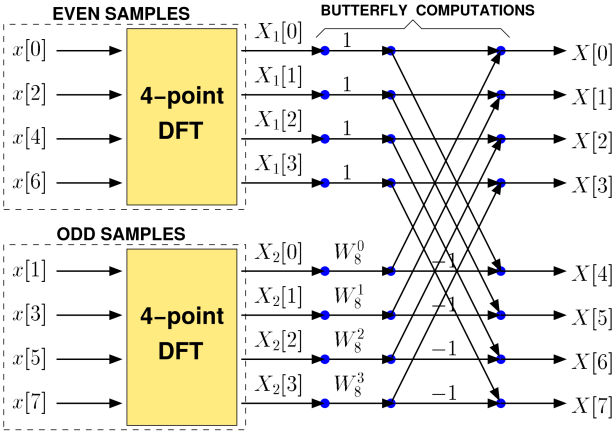




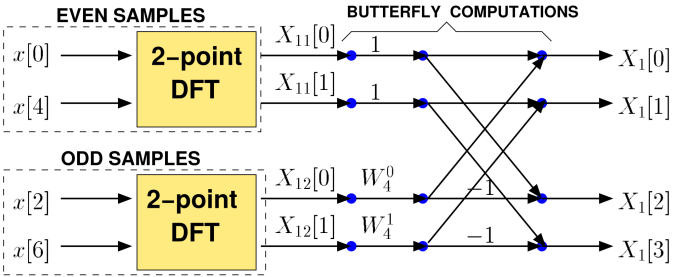
Split $X[k]$ calculation in two:



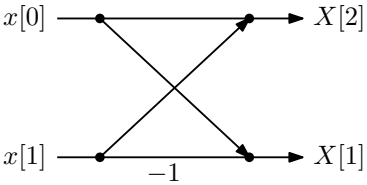
Eight-point DFT



Four-point DFT

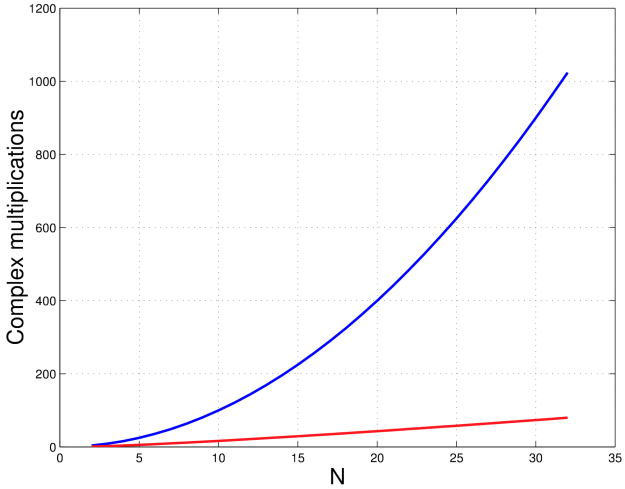


Two-point DFT



$$
\begin{aligned}
x[0] &\longrightarrow X[0] \\
x[4] &\longrightarrow X[1] \\
x[2] &\longrightarrow X[2] \\
x[6] &\longrightarrow X[3] \\
x[1] &\longrightarrow X[4] \\
x[5] &\longrightarrow X[5] \\
x[3] &\longrightarrow X[6] \\
x[7] &\longrightarrow X[7]
\end{aligned}
$$

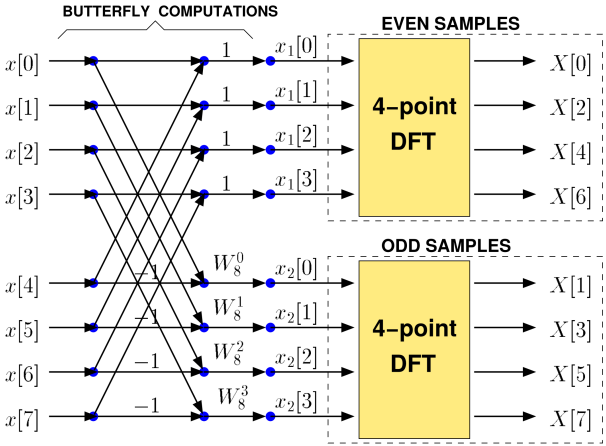
Computational complexity

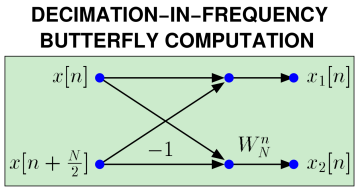


In-place computation of the FFT

Decimal index	Binary index	Bit-reversed index
$x[0]$	$x[000]$	
$x[4]$	$x[100]$	
$x[2]$	$x[010]$	
$x[6]$	$x[110]$	
$x[1]$	$x[001]$	
$x[5]$	$x[101]$	
$x[3]$	$x[011]$	
$x[7]$	$x[111]$	

Decimation in frequency





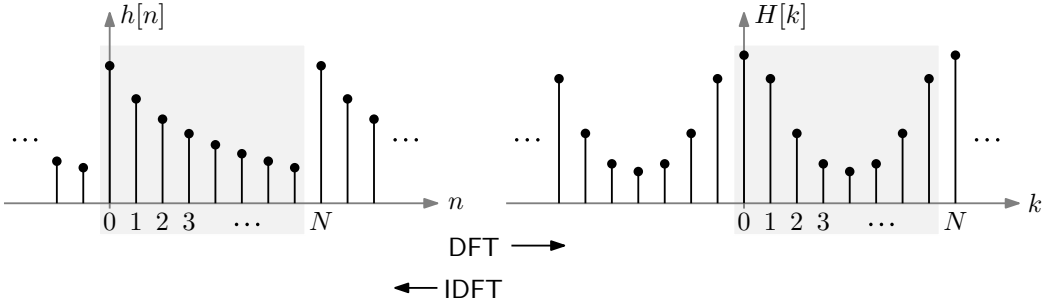
Intermediate summary

Decimation in time:

- Split $x[n]$ into even and odd parts
- Helpers: $X_1[k]$ and $X_2[k]$
- Input: bit-reversed / Output: nice

Decimation in frequency:

- Split $X[k]$ into even and odd parts
- Helpers: $x_1[n]$ and $x_2[n]$
- Input: nice / Output: bit-reversed

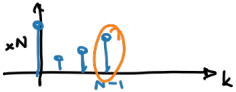

$h[n]$
n
0 1 2 3 $\cdots$ N
$H[k]$
k
0 1 2 3 $\cdots$ N
DFT
IDFT

Calculating IFFTs using FFTs

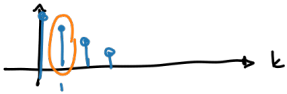
IDFT:

$$x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]e^{j2\pi kn/N}$$

- FFT$\{X[n]\} = N \cdot x[N-k]$



- Swap indices: What happens at N-k now happens at k

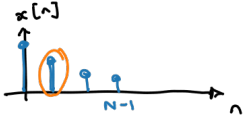


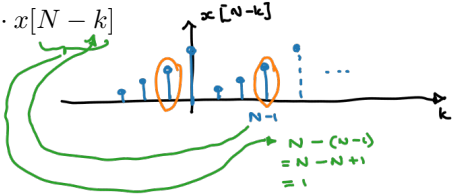
- k becomes n
- Divide by N

Symmetry in DFT:

if $\mathrm{DFT}\{x[n]\} = X[k]$ then

$$\mathrm{DFT}\{X[n]\} = N \cdot x[-k]$$
$$= N \cdot x[N-k]$$





What to take away

- You can now quickly do:
  - DFT using FFT
  - DFT of real sequence using FFT
  - IDFT using FFT
- Some technical tricks
- Divide and conquer: Store intermediate results and use them later
- You need to understand implications of the tricks:
  - When can you (not) use in-place computations?
  - Why radix-2?
  - Why are things slower with `np.fft.fft` when the input is not radix-2?